VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Course: Microprocessors – Microcontrollers (CO3009)
Semester 1, academic year 2026-2027
Ho Chi Minh City, 29/10/2026

ANNOUNCEMENT AND IMPLEMENTATION GUIDE
COURSE PROJECT – MICROPROCESSORS – MICROCONTROLLERS
Topic: Two-Road Traffic Light System on the XNUCLEO-F103RB Board

This document is both the **official announcement** of the Course Project (BTL) and a **step-by-step guide** to help groups get started. **Part A** covers administrative information: groups, deadlines, deliverables and assessment. **Part B** gives technical guidance in the recommended order of work, from receiving the board to submitting the project. Groups should read all of Part A, then work through each step in Part B in order and tick the checklist at the end of each step. This document does not need to be submitted; it is only guidance for the project. If you have any questions, please ask in the course Zalo group. Guide files and reference source code for the project will be uploaded to LMS and to the Zalo group.

Contents

PART A. GENERAL ANNOUNCEMENT 1
A1. Organization 1
A2. Project Requirements 1
A3. Deadlines 1
A4. Deliverables 1
A5. Assessment Criteria (tentative) 1
PART B. IMPLEMENTATION GUIDE 1
B0. Overall Roadmap 1
Step 1. Form a Group and Create a GitHub Repository 1
Step 2. Study the Hardware and Create an STM32CubeIDE Project 1
Step 3. Hardware Testing 1
Step 4. Implement the O(1) Scheduler 1
Step 5. Implement the Supporting Modules 1
Step 6. Design and Implement the State Machines 1
Step 7. System Testing, Report Writing and Submission 1
B8. Common Problems 1
B9. Support Contacts 1

PART A. GENERAL ANNOUNCEMENT

A1. Organization

- The Course Project is carried out in **groups of at most 3 students**.
- Each group uses **a single GitHub repository**. The commit history is the basis for assessing each member's contribution, so **each member must commit using their own account**.

A2. Project Requirements

Each group implements a traffic light system for a two-road intersection on the XNUCLEO-F103RB board (STM32F103RB chip), including:

- **Automatic mode:** the Red – Green – Amber lights of the two roads cycle alternately according to the configured durations.
- **Manual mode:** the user changes the light state with push buttons.
- **Timing adjustment mode:** the Red – Green – Amber durations are changed with push buttons, with the information shown on the LCD.
- **Environment display:** read the DHT20 sensor and show temperature and humidity on the LCD.
- **O(1) scheduler (mandatory):** the whole system runs on a scheduler with SCH_Add_Task, SCH_Delete_Task, SCH_Update (O(1)) and SCH_Dispatch_Tasks; one-shot tasks must be supported.

A3. Deadlines

Milestone	Deadline
Report and source code submission	*[04/12/2026]*
Demo	*[Week 15/16 – per the announced schedule]*

A4. Deliverables

No.	Deliverable	Requirements
1	Report (PDF)	Written using Template_Report_BTL_CO3009.docx; the cover page includes member information (full name, student ID, group) and the GitHub link. File name: CO3009_StudenID1_StudentID2_..._Report.pdf
2	Source code	GitHub link, stating the submitted commit/tag. The repo must be public, or the lecturer must be added as a collaborator. Include a README.md with build and flashing instructions
3	Demo	Live demo on the XNUCLEO-F103RB board; the demo schedule will be announced later

A5. Assessment Criteria (tentative)

No.	Criterion	Weight
1	Scheduler: all functions complete, SCH_Update is O(1), one-shot tasks supported	[10]%
2	State machine design; sensible and safe transitions between modes	[30]%
3	Traffic light functions: automatic, manual, timing adjustment	[20]%
4	Supporting modules: software timer, button, DHT20, LCD	[10]%
5	Code organization: hardware/software layer separation, naming conventions, commit history	[10]%
6	Report and demo	[20]%

Late submissions [penalty rule / not accepted]. Any copying of source code or reports between groups will be handled according to University regulations.

PART B. IMPLEMENTATION GUIDE

B0. Overall Roadmap

The project should be built layer by layer, from the bottom up: **hardware works correctly → scheduler works correctly → supporting modules work correctly → state machines**. Each layer should be tested independently before building the next one. Catching bugs early in the lower layers will save a great deal of debugging time later.

Week	Tasks	Expected outcome
Week 1	Steps 1 – 3: form group, create repo, create project, test hardware	LED blinking, 4 light states, 4 buttons, I2C scan finds 2 devices
Week 2	Step 4: implement and test the scheduler	Multiple tasks run at the correct periods; one-shot tasks delete themselves
Week 3	Step 5: software timer, button, lights, DHT20, LCD	Each module has a small demo running on the scheduler
Week 4	Step 6: design and implement the 3 state machines	Complete system
Week 5	Step 7: system testing, report writing, demo preparation	PDF report, submission tag

[Guide] *This roadmap is only a suggestion; groups should adjust it to the milestones in section A3.*

Step 1. Form a Group and Create a GitHub Repository

The group leader creates the repository and adds the members as collaborators; each member then clones it to their machine:

```
git clone https://github.com/<account>/<repo-name>.git
cd <repo-name>
git checkout -b dev            # shared development branch
git checkout -b feature/scheduler   # one branch per piece of work
```

Add a **.gitignore** file in the root folder so that build-generated files are not pushed to GitHub:

```
Debug/
Release/
*.o
*.d
*.elf
*.map
*.su
.metadata/
```

- Agree on naming conventions from the start (e.g. lowercase file names joined with underscores: fsm_automatic.c; function names in the form <module>_<action>: button_read(), traffic_set_state()).

- Commit frequently, one meaningful change per commit, with clear messages (e.g. "scheduler: add one-shot task support").
- Reference project structure: https://github.com/nhanksd85/STM32F103_MCU/blob/scheduler

Checklist for completing this step

- ☐ Repo created with all members, .gitignore and README.md
- ☐ Group and GitHub link registered on LMS
- ☐ Group has agreed on naming conventions and task assignment

Step 2. Study the Hardware and Create an STM32CubeIDE Project

2.1 The XNUCLEO-F103RB Board

The project is developed on the XNUCLEO-F103RB board (STM32F103RB chip). This is important information when creating the project in STM32CubeIDE.

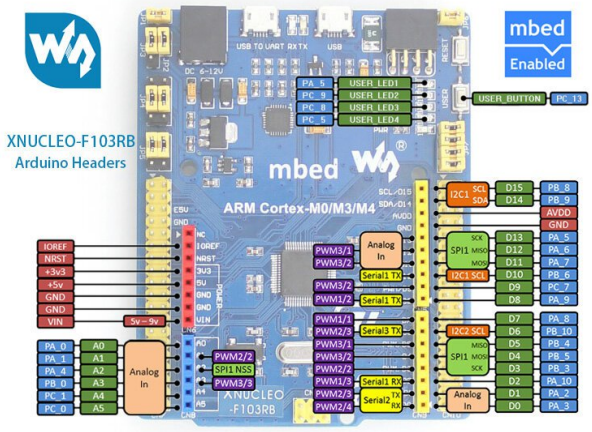


Figure 1: The XNUCLEO-F103RB board

The pins on the Arduino-compatible headers are named **Dx** (where x is the pin number). Each Dx pin is connected to an STM32 pin; for example, D2 is connected to PA10. Students must **look up the schematic themselves** below to find the corresponding STM32 pins.

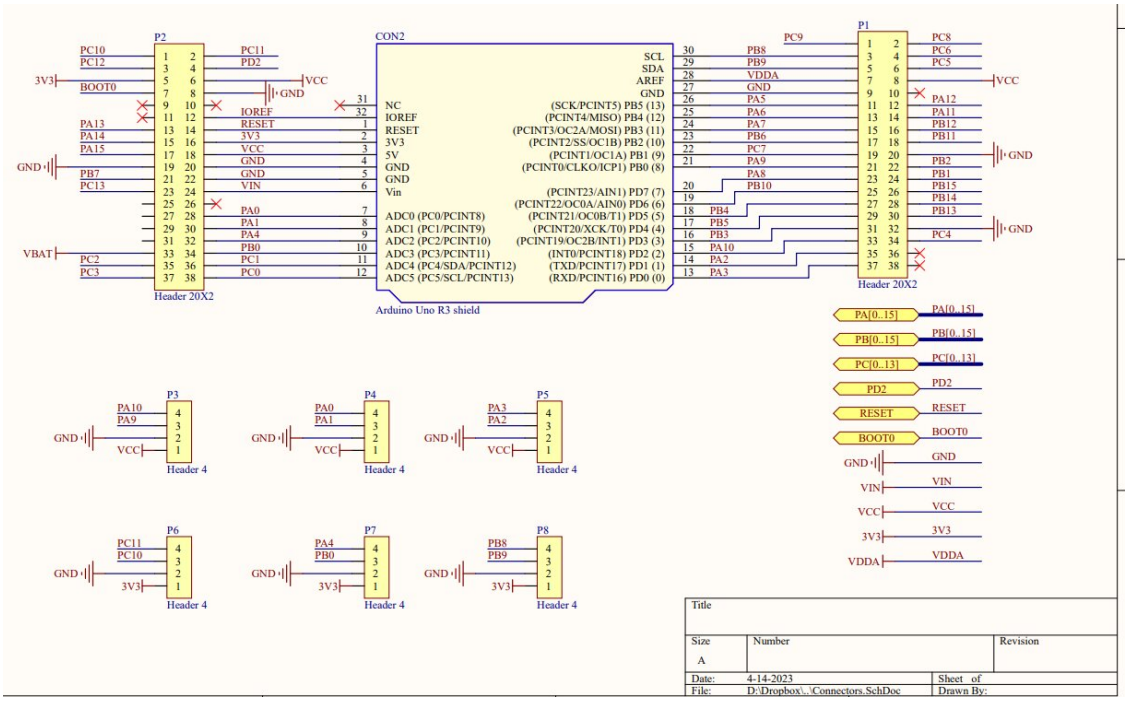


Figure 2: Schematic of the Arduino headers

The XNUCLEO board is fitted with a GROVE-standard pin expansion board. Each GROVE slot has 2 signal pins.

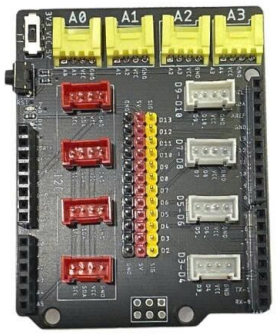


Figure 3: GROVE-standard pin expansion board

2.2 Connecting the Devices

Device	Connection	Notes
Traffic light module 1	D3 – D4	2 signals → 4 states
Traffic light module 2	D5 – D6	2 signals → 4 states
Button module 1	D7 – D8	2 buttons, wired directly to GPIO
Button module 2	D9 – D10	2 buttons, wired directly to GPIO
DHT20 sensor, LCD	I2C slot (red)	Any slot can be used
On-board LEDs / USER button	PA5, PC9 / PC13	Built into the board

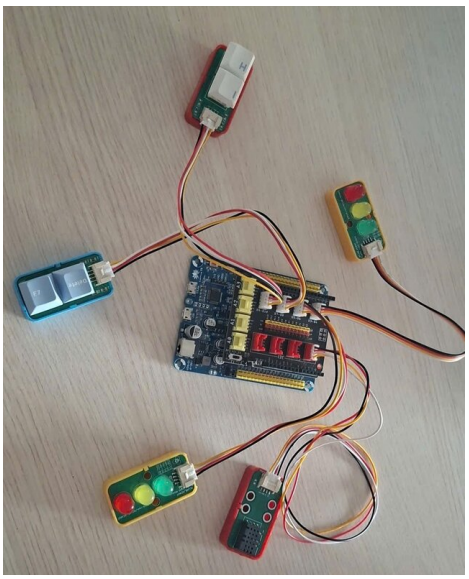


Figure 4: The fully connected system

Using the schematic, fill in the following pin mapping table. This table is reused in section 3.1 of the report.

Arduino pin	STM32 pin	GPIO mode	User Label in .ioc
D3	*[Pxx]*	Output Push-Pull	*[...]*
D4	*[Pxx]*	Output Push-Pull	*[...]*
D5	*[Pxx]*	Output Push-Pull	*[...]*
D6	*[Pxx]*	Output Push-Pull	*[...]*
D7	*[Pxx]*	Input	*[...]*
D8	*[Pxx]*	Input	*[...]*
D9	*[Pxx]*	Input	*[...]*
D10	*[Pxx]*	Input	*[...]*

2.3 Creating the Project in STM32CubeIDE

> **IMPORTANT NOTE**
> The XNUCLEO-F103RB uses the **STM32F103RB** chip, which differs from the STM32F103C6 used in the simulation environment. If it is configured incorrectly, the board **can be flashed only once** and cannot be flashed again afterwards. Follow the two steps below exactly.

Step 2a: Create a new project, switch to the **Board Selector** tab, then search for and select the board **NUCLEO-F103RB**.

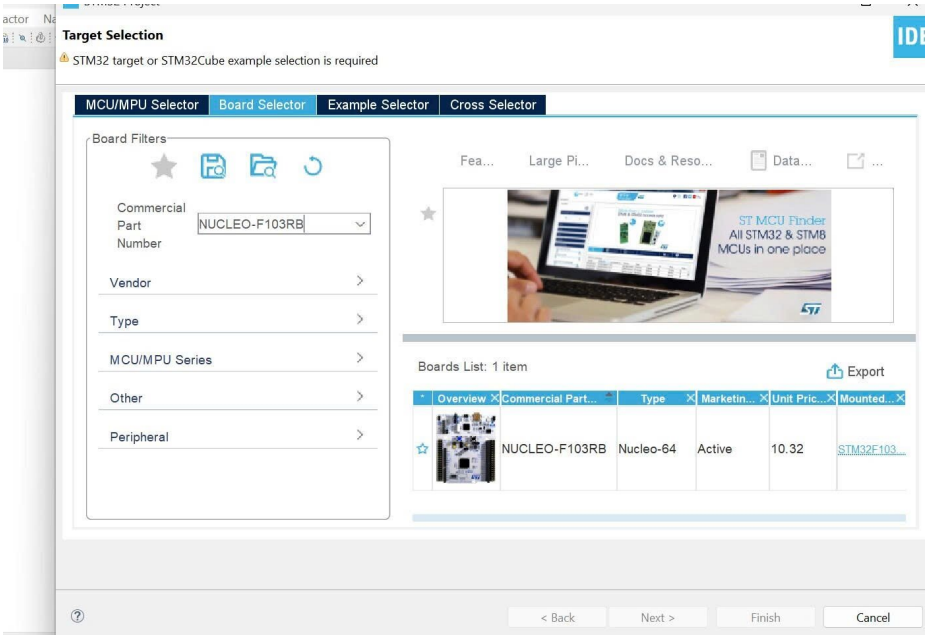


Figure 5: Selecting the NUCLEO-F103RB board in Board Selector

Step 2b: Open the .ioc file and go to **Pinout & Configuration → System Core → SYS**. Under **Debug** you must select **Serial Wire**.

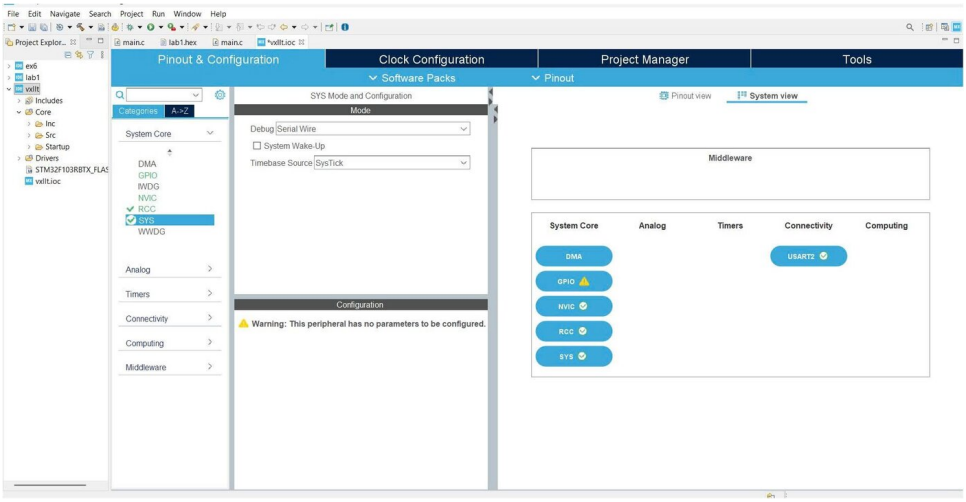


Figure 6: Setting Debug to Serial Wire

If the board can no longer be flashed, redo both steps above, rebuild the program and follow this video: https://www.youtube.com/watch?v=qxaW6FMiVWQ. If it still cannot be flashed, contact the lecturer for support.

2.4 Initial Configuration in the .ioc File

- Configure the GPIO pins according to the pin mapping table in section 2.2 and give them meaningful User Labels (e.g. TL1_A, TL1_B, BTN_1…).
- Enable **I2C1** for the DHT20 and LCD (check on the schematic which SCL/SDA pin pair the red I2C slot is connected to).
- Configure a timer (e.g. TIM2) to generate a periodic interrupt every **10 ms** as the tick for the scheduler, and enable the timer interrupt in the NVIC.

The interrupt period is given by T = (PSC + 1) × (ARR + 1) / f_TIM. For example, with f_TIM = 8 MHz, choosing PSC = 7999 and ARR = 9 gives T = 10 ms. Groups must check the **actual f_TIM** in the project's Clock Configuration tab and recalculate PSC and ARR accordingly.

Checklist for completing this step

- ☐ Project created from Board Selector with NUCLEO-F103RB, Debug = Serial Wire
- ☐ Pin mapping table for D3 – D10 fully filled in
- ☐ GPIO, I2C1 and 10 ms timer configured; project builds without errors
- ☐ Base project committed to GitHub

Step 3. Hardware Testing

Before writing the scheduler or the state machines, the group must make sure every device works correctly. The test code in this step is temporary and is written directly in the while(1) loop of main.c.

3.1 Checking Program Flashing: Blink LED PA5

```
while (1) {
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    HAL_Delay(500);
}
```

3.2 Determining the Signal Combinations for the Light Modules

Each light module is controlled by 2 pins, giving 4 states: Green, Amber, Red and Off. Output the 4 combinations one by one, observe, and fill in the table:

```
// Try the 4 combinations (0,0) (0,1) (1,0) (1,1) in turn, holding each for 2 seconds
for (int i = 0; i < 4; i++) {
    HAL_GPIO_WritePin(TL1_A_GPIO_Port, TL1_A_Pin, (i >> 1) & 1);
    HAL_GPIO_WritePin(TL1_B_GPIO_Port, TL1_B_Pin, i & 1);
    HAL_Delay(2000);
}
```

Signal A	Signal B	Observed light state
0	0	*[...]*
0	1	*[...]*
1	0	*[...]*
1	1	*[...]*

3.3 Testing the Buttons

Read each button and show the result on LED PA5 to determine the logic level when pressed (0 or 1). This information is needed for the button module in Step 5.

```
if (HAL_GPIO_ReadPin(BTN_1_GPIO_Port, BTN_1_Pin) == GPIO_PIN_RESET)
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
else
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
```

3.4 Scanning I2C Devices

Scan all I2C addresses to confirm that the DHT20 and LCD are connected, and to find the LCD's address (the LCD address varies between modules). Set a breakpoint or use the LED to view the result.

```
uint8_t found[8]; int n = 0;
for (uint8_t addr = 1; addr < 128; addr++) {
    if (HAL_I2C_IsDeviceReady(&hi2c1, addr << 1, 1, 10) == HAL_OK)
        if (n < 8) found[n++] = addr;   // set a breakpoint to inspect the found array
}
```

Checklist for completing this step

- ☐ LED PA5 blinks; the program can be re-flashed many times without errors
- ☐ Light signal combination table determined for both modules
- ☐ Logic level when pressed known for all 4 buttons and the USER button
- ☐ I2C scan finds 2 devices; DHT20 and LCD addresses recorded

Step 4. Implement the O(1) Scheduler

The scheduler is **mandatory** and is the foundation of the whole system. From this step on, **do not use HAL_Delay** in the main program; all work is run as tasks.

4.1 Data Structure and Functions

```
typedef struct {
    void   (*pTask)(void);   // pointer to the task function
    uint32_t Delay;           // remaining ticks (relative)
    uint32_t Period;          // period, 0 = one-shot task
    uint8_t  RunMe;           // flag: task ready to run
    uint32_t TaskID;
} sTask;

void     SCH_Init(void);
uint32_t SCH_Add_Task(void (*pFunc)(void), uint32_t DELAY, uint32_t PERIOD);
uint8_t  SCH_Delete_Task(uint32_t taskID);
void     SCH_Update(void);          // called in the timer interrupt
void     SCH_Dispatch_Tasks(void);  // called in while(1)
```

SCH_Update must have a complexity of **O(1)**: it only processes the task at the head of the list.

```
void SCH_Update(void) {
    if (SCH_count == 0) return;
    if (SCH_tasks_G[0].Delay > 0) SCH_tasks_G[0].Delay--;
    if (SCH_tasks_G[0].Delay == 0) SCH_tasks_G[0].RunMe = 1;
}
```

> **KEY IDEA**
> For SCH_Update to only need to check the first task, the task list must be **sorted by run time**, and each task stores a **relative Delay** with respect to the task before it. The heavy work is then moved into SCH_Add_Task: find the correct insertion point, subtract the total Delay of the

> preceding tasks from the new task's Delay, and update the Delay of the task immediately after it.
>
> In SCH_Dispatch_Tasks: when the head task has RunMe = 1, call the task function and remove the task from the head of the list; if Period > 0, re-add it with Delay = Period; if Period = 0 (one-shot), do not re-add it.
>
> Each group chooses an Array or Linked List implementation and justifies the choice in the report.

4.2 Testing the Scheduler

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) SCH_Update();
}

// in main(), before while(1):
SCH_Init();
SCH_Add_Task(toggle_PA5, 0, 50);     // 500 ms (tick 10 ms)
SCH_Add_Task(toggle_PC9, 0, 100);    // 1000 ms
SCH_Add_Task(one_shot_test, 300, 0); // runs once after 3 s
while (1) { SCH_Dispatch_Tasks(); }
```

Checklist for completing this step

- ☐ Both LEDs blink with the correct periods of 0.5 s and 1 s when running together
- ☐ The one-shot task runs exactly once and is removed from the list
- ☐ SCH_Delete_Task can delete any task while the remaining tasks keep correct timing
- ☐ Tasks have been added with many different Delay values (inserted at the head, middle and tail of the list)

Step 5. Implement the Supporting Modules

The modules in this step are usually **hardware-dependent** and are the only place where HAL functions may be called. Each module consists of a .h/.c file pair and provides functions for the state machine layer to use.

Module	Suggested files	Responsibilities and notes
Software timer	software_timer.h/.c	Timers used by the state machines (setTimer, timeout flags); updated by a task with a 10 ms period
Button	button.h/.c	Reads buttons every 10 ms, debounces (compares several consecutive readings), detects presses and long presses; provides functions such as isButtonPressed(i)
Traffic lights	traffic_light.h/.c	Functions to set the state of each road, e.g. traffic_set(1, RED), using the combination table from section 3.2
DHT20	dht20.h/.c	I2C communication; reads and converts temperature and humidity per the datasheet; reads at a reasonable period (not continuously)
LCD	lcd.h/.c	Initialize, clear screen, set cursor, print strings/numbers; uses the address found in section 3.4

Each module should have a small test program running on the scheduler. For example: pressing button 1 toggles the LED; every 2 seconds, read the DHT20 and print to the LCD.

Checklist for completing this step

- ☐ A single button press is not registered multiple times due to bouncing
- ☐ Each road's lights can be controlled through the light module's functions
- ☐ LCD shows temperature and humidity, updated periodically
- ☐ No HAL calls remain outside the supporting modules and main.c

Step 6. Design and Implement the State Machines

6.1 Design First, Code Later

Draw the state machines on paper or with a drawing tool (draw.io, Graphviz…) and agree on them within the group before writing code. Suggested division into 3 main state machines:

- **fsm_automatic:** the combined states of the 2 roads (e.g. Red1–Green2, Red1–Amber2, Green1–Red2, Amber1–Red2); transitions occur when a software timer expires.
- **fsm_manual:** the user changes the light state with push buttons.
- **fsm_tuning:** select the light to adjust, increase/decrease the duration, confirm; the LCD shows the value being adjusted.

The LCD can be used as a menu to display and select modes.

> **KEY FOCUS OF ASSESSMENT**
>
> For each transition between the main modes (e.g. from automatic to manual), the group must be able to answer: what state the lights of both roads are in right after the transition; how timers and tasks are re-initialized; what the LCD shows; and how to ensure that **the two roads are never Green at the same time**.
>
> When adjusting durations, valid constraints between the Red, Green and Amber durations must be defined, as well as how to handle invalid values entered by the user.

6.2 Implementation Rules

- Each state machine is its own .h/.c file pair; the function that runs the state machine is called as a scheduler task.
- The state machines **must NOT contain hardware-access calls** (HAL_GPIO_…, HAL_I2C_…). All operations on lights, buttons and the LCD go through the functions of the modules from Step 5.
- Shared variables (current state, Red/Green/Amber durations) are declared in global.h/global.c.

```
/* fsm_automatic.c – suggested skeleton */
#include "fsm_automatic.h"

void fsm_automatic_run(void) {
    switch (status) {
    case AUTO_RED1_GREEN2:
        traffic_set(1, RED); traffic_set(2, GREEN);
        if (timer_flag[0] == 1) {
            status = AUTO_RED1_AMBER2;
            setTimer(0, time_amber);
        }
        if (isButtonPressed(0))  { status = MAN_INIT; }
        break;
    // TODO: remaining states
    default: break;
    }
}
```

Suggested folder structure:

```
Core/Inc/   global.h  scheduler.h  software_timer.h  button.h
            traffic_light.h  dht20.h  lcd.h
            fsm_automatic.h  fsm_manual.h  fsm_tuning.h
Core/Src/   corresponding .c files
```

Checklist for completing this step

- ☐ Diagrams of the 3 state machines and the mode-switching state machine are complete
- ☐ System runs all 3 modes and switches modes as designed
- ☐ No HAL calls in the fsm_*.c files
- ☐ Edge cases tested: repeated button presses, switching mode while a light is Amber, entering invalid durations

Step 7. System Testing, Report Writing and Submission

1. Run the system continuously for at least [..] minutes to check stability.
2. Write the report using Template_Report_BTL_CO3009.docx; delete the grey guidance paragraphs before exporting to PDF.
3. Update README.md: project description, folder structure, how to build and flash, how to operate.
4. Merge the source code into the main branch, add a submission tag and write the commit hash on the report's cover page:

```
git checkout main
git merge dev
git tag v1.0-final
git push origin main --tags
```

5. Submit the file CO3009_Nhom<XX>_Report.pdf and the GitHub link on LMS before the deadline.

B8. Common Problems

Symptom	Solution
Flashes once, then cannot be flashed again	Check Board Selector (NUCLEO-F103RB) and SYS → Debug = Serial Wire; follow the recovery video in section 2.3
Tasks do not run or run with the wrong period	Check that HAL_TIM_Base_Start_IT is called, the timer interrupt is enabled in the NVIC, and PSC/ARR are correct for f_TIM
Program hangs after a while	Check for task array overflow, tasks being added repeatedly without being deleted, or leftover HAL_Delay calls in tasks
One button press is registered multiple times	Review the debouncing mechanism and clearing of the button flag after handling
LCD shows nothing	Re-run the I2C scan to confirm the address; remember to shift the 7-bit address left by 1 bit when calling HAL functions
Lights show the wrong colors	Re-check the signal combination table in section 3.2 and the pin order in the .ioc

B9. Support Contacts

For any questions about the content, students should post on the course forum so that other groups can benefit. If a board can no longer be flashed or the hardware is damaged, contact the lecturer directly.

Attachments: Project_Group.docx (detailed specification), Template_Report_BTL_CO3009.docx (report template).